Architecture

```
src/
  routes/
    __root.tsx          root layout, fonts, global meta, toaster
    index.tsx           website home
    menu.tsx            public menu (reads the same item data as billing)
    branches.tsx        3 branches
    contact.tsx         contact / bulk orders
    app.tsx             owner-app layout: header, branch selector, tab nav, <Outlet/>
    app.index.tsx       dashboard (KPIs, charts, channel P&L)
    app.billing.tsx     POS billing + thermal receipt printing
    app.reports.tsx     item-wise report, prep forecast, CSV export
    app.stock.tsx       stock on hand, movements, add material
    app.items.tsx       menu & cost editor
    app.settings.tsx    business, channels, branches, backup/restore
  components/site/SiteShell.tsx   public header + footer
  lib/pos-store.ts      types, seed data, storage, profit maths, forecast
  styles.css            design tokens (colours, fonts, shadows)
  assets/               food photography
```

Routing is file-based (TanStack Router). A file named `app.billing.tsx` serves `/app/billing`.

`src/routeTree.gen.ts` is generated - never edit it.

Data layer (`src/lib/pos-store.ts`)

One JSON document in `localStorage` under `slb.pos.v1`:

```
DB = { branches, channels, items, sales, stock, moves, settings }
```

API used by every screen:

| Function | Purpose |

| --- | --- |

| `useDB()` | subscribe to the store; returns `null` during SSR, data after hydration |

| `setDB(mutate)` | mutate + save + notify all screens |

| `buildSale({...})` | the single source of truth for profit maths |

| `filterSales`, `totals`, `itemBreakdown`, `dailySeries` | reporting helpers |

| `forecastQty(sales, itemId, date)` | weighted average of last 4 same weekdays |

| `exportJSON` / `importJSON` / `resetDB` | backup, restore, demo reset |

Screens never touch `localStorage` directly, which is why the cloud upgrade is contained.

Moving to Cloud (multi-branch, logins, one shared database)

1. Enable Lovable Cloud (Supabase under the hood) in the project.

2. Create tables mirroring the types: `branches`, `channels`, `items`, `sales`,

`sale_lines`, `stock_items`, `stock_moves`, `settings`, plus `profiles` and a

separate `user_roles` table (roles must never live on the profile row).

3. Enable RLS on every table; scope rows by branch and role (`owner` sees all branches, `cashier` sees their own).

4. Replace the body of `useDB`/`setDB` with TanStack Query hooks calling server functions. UI components stay unchanged because they only use the store API.

5. Keep writing bills to localStorage first and syncing when online - that gives offline billing, which is a genuine selling point in Chennai power cuts.

Ballpark effort: 2-4 focused days.

Why localStorage first (talking point for the owner)

Zero monthly cost, works with no internet, nothing to log into, and no data leaves his shop. When he opens a 4th branch or wants to check sales from home, the cloud upgrade becomes worth paying for.